

Funding Navigator AI – Observations & Analysis

RAG Pipeline • FastAPI • Milvus • LangChain • Ollama • DeepEval

1. LLM Model Used

Model: LLaMA 3 8B Instruct (Ollama)

Purpose

- Used within the RAG workflow for answer generation and experimentation.
- Temperature tuning was performed to study response quality.

Observations

- Lower temperatures produced more deterministic responses.
- Medium temperatures provided a good balance between factual accuracy and explanation during experimentation.
- The model occasionally over-explained responses and struggled to combine information spread across multiple retrieved chunks.

Characteristics

- Transformer-based instruction-tuned model
- Runs locally using Ollama
- Generates responses from retrieved context in a RAG pipeline

Limitations

- Answer quality depends heavily on retrieved context.
- Multi-chunk synthesis can still be improved.

2. Chunking Strategy

RecursiveCharacterTextSplitter

- chunk_size = 1000
- chunk_overlap = 200

Observations

- Preserved semantic continuity using overlap.
- Paragraph-level splitting produced better retrieval quality.
- Structured lists and long policy documents could still be fragmented across chunks.

3. Embeddings & Indexing

Embedding Model: SentenceTransformer (all-MiniLM-L6-v2)

Vector Database: Milvus

Index

- HNSW
- COSINE similarity

Why HNSW?

- Fast approximate nearest-neighbour retrieval
- High retrieval accuracy
- Suitable for semantic search

Metadata

- language
- is_active
- crawl_date
- doc_last_modified

4. FastAPI Backend

The retrieval pipeline is exposed through a FastAPI REST API.

Endpoints

- GET /
- POST /search
- POST /batch_search

Observations

- FastAPI automatically generates Swagger documentation (/docs).
- Returns structured JSON containing:
 - question
 - predicted_answer
 - relevant_chunks
 - similarity score
 - distance
- Simplifies testing and integration.

5. Retrieval Quality

- Retrieval precision improved after complete embedding ingestion.
- Retrieval quality depends on preprocessing, chunking and embedding quality.
- Complex queries may require better multi-chunk aggregation.

6. Evaluation

DeepEval Metrics

- Answer Relevancy
- Faithfulness
- Contextual Precision
- Contextual Recall

Key Findings

- High contextual precision.
- Retrieval performance was stronger than generation quality.
- Prompt engineering and reranking can improve answer quality.

7. Lessons Learned

- Better preprocessing improves retrieval.
- Chunk quality directly impacts semantic search.
- Strong retrieval does not always guarantee perfect answer generation.
- Prompt engineering remains important.

8. Future Improvements

- Streamlit frontend
- Hybrid search
- Reranking
- Better structured document handling
- Cloud deployment

9. Conclusion

Strengths

- Strong semantic retrieval using Milvus
- FastAPI backend
- Modular RAG pipeline
- High contextual precision

Areas for Improvement

- Better multi-chunk synthesis
- Improve answer relevancy and faithfulness
- Expand dataset coverage